IF2211: Laporan Tugas Kecil 3

Ice Sliding Puzzle Solver

Dipersiapkan oleh:

13524138 Ahmad Rinofaros Muchtar

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2025

Daftar Isi

Algoritma 1
Analisis 1
Source Program 3
Tangkapan Layar 25
Analisis Hasil 30
Lampiran 31

Bab 1

Algoritma

a. Uniform Cost Search: Mencari jalur dengan biaya kumulatif terendah tanpa informasi perkiraan jarak ke tujuan
b. Greedy Best-First Search: Mencari jalur dengan memprioritaskan simpul yang memiliki nilai heuristik biaya terkecil
c. A*: Mencari jalur dengan mempertimbangkan biaya aktual dari titik awal dan estimasi biaya ke titik tujuan
d. Langkah Algoritma:
    1. Inisialisasi: Program membuat antrian prioritas yang menyimpan simpul yang akan dieksplorasi. simpul awal dimasukkan dengan biaya 0.
    2. Ekstraksi: Program mengambil simpul dengan prioritas tertinggi (nilai (f(n) terkecil) dari antrean
    3. Pengecekan Tujuan: Program memeriksa apakah posisi saat omo adalah target tujuan dan apakah semua digit angka yang ada di peta sudah dilewati sesuai urutan
    4. Ekspansi: Dari posisi saat ini, program mencoba bergerak ke empat arah(Right, Left, Up, Down). Karakter akan bergerak hingga menabrak dinding X atau batas petak
    5. Perhitungan Biaya: Selama bergerak, biaya setiap langkah dijumlahkan berdasarkan matriks biaya yang diberikan
    6. Pembaruan Jalur: Jika terdapat jalur ke suatu posisi simpul baru yang lebih murah,, program akan memperbarui penjumlahan biaya terendah dan memasukkan posisi simpul tersebut ke antrean prioritas
    7. Pengulangan: Hingga tujuan ditemukan atau antrian kosonh

Bab 2

Analisis

a. Definisi dari f(n) dan g(n) pada algoritma.

1. g(n): Biaya aktual yang ditempuh dari titik awal ke posisi n saat ini. Disimpan dalam variabel moveCost dan merupakan hasil dari fungsi Slide
2. f(n): Fungsi evaluasi total yang menentukan urutan ekspolarasi dalam PriorityQueue
    a. Pada UCS: f(n) = g(n)
    b. Pada GBFS: f(n) = h(n)
    c. Pada A*: f(n) = g(n) + h(n)

b. Apakah heuristik yang digunakan pada algoritma A* admissible?
   Ya, fungsi heuristik dalam program bersifat permissible karena tidak memberikan nilai yang lebih besar(overestimate) dari biaya sebenarnya. Hal ini karena biaya setiap langkah di grid minimal 1 atau sesuai matriks biaya, sehingga jarak geometris selalu menjadi batas bawah(lower bound) dari biaya nyata.
c. Secara teoritis, perbandingan algoritma yang diimplementasikan pada permasalahan ini:
    1. UCS: Menjamin solusi optimal, namun tidak efisien secara memori dan waktu
    2. GBFS: Sangat cepat dan efisien karena mengarah ke target, namun solusi optimal tidak terjamin
    3. A*: Menggabungkan UCS dan GBFS sehingga dapat menemukan jalur optimal dengan jumlah iterasi lebih sedikit dibanding

Bab 3

Source Program

a. AlgorithmKind.java

```
public enum AlgorithmKind {
  UCS,
  GBFS,
  ASTAR
}
```

b. HeuristicKind.java

```
public enum HeuristicKind {
  H1,
  H2,
  H3
}
```

c. IceLogic.java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public final class IceLogic {

  private static final int[][] DELTAS = {{0, 1}, {0, -1}, {-1, 0},
{1, 0}};
  private static final char[] MOVE_NAME = {'R', 'L', 'U', 'D'};

  private IceLogic() {}

  public static char moveName(int dirIndex) {
    return MOVE_NAME[dirIndex];
  }

  public static int numDirections() {
```

```
    return 4;
  }

  //Kembalikan null jika invalid
  public static SlideResult slide(
      String[] grid,
      int[][] costs,
      int r,
      int c,
      int dirIndex,
      int phase,
      int[] goals,
      int n,
      int m) {
    int dr = DELTAS[dirIndex][0];
    int dc = DELTAS[dirIndex][1];
    int p = phase;
    int cr = r, cc = c;
    long moveCost = 0;
    int tr = cr + dr, tc = cc + dc;
    if (tr < 0 || tr >= n || tc < 0 || tc >= m) return null;

    while (true) {
      if (tr < 0 || tr >= n || tc < 0 || tc >= m) return null;
      char ch = grid[tr].charAt(tc);
      if (ch == 'X') break;
      if (ch == 'L') return null;
      if (ch >= '0' && ch <= '9') {
        int d = ch - '0';
        if (p < goals.length) {
          int need = goals[p];
          if (d > need) return null;
          if (d == need) p++;
        }
      }
      moveCost += costs[tr][tc];
      cr = tr;
      cc = tc;
      tr += dr;
```

```
      tc += dc;
    }
    if (cr == r && cc == c) return null;
    if (moveCost > Integer.MAX_VALUE) throw new
IllegalStateException("Cost overflow");
    return new SlideResult(cr, cc, p, (int) moveCost);
  }

  public static int minPositiveTraversableCost(String[] grid,
int[][] costs) {
    Integer best = null;
    int n = grid.length, m = grid[0].length();
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
        char ch = grid[i].charAt(j);
        if (ch == '*' || ch == 'O' || (ch >= '0' && ch <= '9')) {
          int v = costs[i][j];
          if (v > 0 && (best == null || v < best)) best = v;
        }
      }
    }
    return best == null ? 1 : best;
  }

  /** Digit -> (row,col) for digits present on grid. */
  public static Map<Integer, int[]> digitPositions(String[] grid)
{
    Map<Integer, int[]> map = new HashMap<>();
    int n = grid.length, m = grid[0].length();
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
        char ch = grid[i].charAt(j);
        if (ch >= '0' && ch <= '9') map.put(ch - '0', new int[]
{i, j});
      }
    }
    return map;
  }
```

```java
  public static double heuristic(
      HeuristicKind h,
      int r,
      int c,
      int phase,
      int[] goals,
      Map<Integer, int[]> digitPos,
      int gr,
      int gc,
      int minPosCost) {
    switch (h) {
      case H1:
        return (Math.abs(r - gr) + Math.abs(c - gc)) * (double)
minPosCost;
      case H2:
        {
          int tr, tc;
          if (phase < goals.length) {
            int[] xy = digitPos.get(goals[phase]);
            tr = xy[0];
            tc = xy[1];
          } else {
            tr = gr;
            tc = gc;
          }
          return (Math.abs(r - tr) + Math.abs(c - tc)) * (double)
minPosCost;
        }
      case H3:
        return Math.max(Math.abs(r - gr), Math.abs(c - gc)) *
(double) minPosCost;
      default:
        throw new IllegalArgumentException("Unknown heuristic");
    }
  }

  public static List<String[]> applyMoves(ProblemInstance p,
String path) {
    int n = p.n(), m = p.m();
```

```
    int zr = p.startR, zc = p.startC;
    int phase = 0;
    List<String[]> frames = new ArrayList<>();
    frames.add(render(p.grid, p.goalR, p.goalC, zr, zc, phase,
p.goals, n, m));
    for (int k = 0; k < path.length(); k++) {
      char mv = path.charAt(k);
      int di = moveIndex(mv);
      if (di < 0) break;
      SlideResult res = slide(p.grid, p.costs, zr, zc, di, phase,
p.goals, n, m);
      if (res == null) break;
      zr = res.endR;
      zc = res.endC;
      phase = res.newPhase;
      frames.add(render(p.grid, p.goalR, p.goalC, zr, zc, phase,
p.goals, n, m));
    }
    return frames;
  }

  private static int moveIndex(char mv) {
    if (mv == 'R') return 0;
    if (mv == 'L') return 1;
    if (mv == 'U') return 2;
    if (mv == 'D') return 3;
    return -1;
  }

  private static String[] render(
      String[] grid,
      int gr,
      int gc,
      int zr,
      int zc,
      int phase,
      int[] goals,
      int n,
      int m) {
```

```
    char[][] g2 = new char[n][m];
    for (int i = 0; i < n; i++) g2[i] = grid[i].toCharArray();
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
        char ch = g2[i][j];
        if (ch >= '0' && ch <= '9') {
          int d = ch - '0';
          if (phase >= goals.length) g2[i][j] = '*';
          else {
            int need = goals[phase];
            if (d < need) g2[i][j] = '*';
          }
        }
      }
    }
    g2[gr][gc] = 'O';
    g2[zr][zc] = 'Z';
    String[] out = new String[n];
    for (int i = 0; i < n; i++) out[i] = new String(g2[i]);
    return out;
  }

  public static String[] renderExplorationState(
      String[] grid, int gr, int gc, State st, int[] goals, int n,
int m) {
    return render(grid, gr, gc, st.r, st.c, st.phase, goals, n,
m);
  }
}
```

d. IceSearcher.java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
```

```java
public final class IceSearcher {

  private static final double EPS = 1e-9;

  private IceSearcher() {}

  public static SearchOutcome search(ProblemInstance p,
AlgorithmKind algo, HeuristicKind heur) {
    String[] grid = p.grid;
    int[][] costs = p.costs;
    int n = p.n(), m = p.m();
    Map<Integer, int[]> digitPos = IceLogic.digitPositions(grid);
    int minPc = IceLogic.minPositiveTraversableCost(grid, costs);
    int[] goals = p.goals;
    int goalR = p.goalR, goalC = p.goalC;

    State start = new State(p.startR, p.startC, 0);

    Map<State, MoveParent> parents = new HashMap<>();
    parents.put(start, new MoveParent(null, null));

    Map<State, Double> gScore = new HashMap<>();
    gScore.put(start, 0.0);

    int tieCounter = 0;
    PriorityQueue<PQNode> pq =
        new PriorityQueue<>(
            Comparator.comparingDouble((PQNode x) ->
x.priority).thenComparingInt(x -> x.tie));

    pq.add(
        new PQNode(priority(algo, heur, goals, digitPos, minPc,
goalR, goalC, start, 0.0), tieCounter++, start, 0.0));

    List<State> exploration = new ArrayList<>();
    int iterations = 0;
    long t0 = System.nanoTime();
    State goalState = null;
```

```
    while (!pq.isEmpty()) {
      iterations++;
      PQNode node = pq.poll();
      State st = node.state;
      double g = node.g;
      double bestG = gScore.getOrDefault(st,
Double.POSITIVE_INFINITY);
      if (g > bestG + EPS) continue;

      exploration.add(st);
      if (st.r == goalR && st.c == goalC && st.phase >=
goals.length) {
        goalState = st;
        break;
      }

      for (int d = 0; d < IceLogic.numDirections(); d++) {
        SlideResult res = IceLogic.slide(grid, costs, st.r, st.c,
d, st.phase, goals, n, m);
        if (res == null) continue;
        double ng = g + res.moveCost;
        State next = new State(res.endR, res.endC, res.newPhase);
        if (ng + EPS < gScore.getOrDefault(next,
Double.POSITIVE_INFINITY)) {
          gScore.put(next, ng);
          char mv = IceLogic.moveName(d);
          parents.put(next, new MoveParent(st, mv));
          double pri =
              priority(algo, heur, goals, digitPos, minPc, goalR,
goalC, next, ng);
          pq.add(new PQNode(pri, tieCounter++, next, ng));
        }
      }
    }

    double elapsedMs = (System.nanoTime() - t0) / 1_000_000.0;

    if (goalState == null) return new SearchOutcome(null, 0,
iterations, elapsedMs, exploration);
```

```
    StringBuilder path = new StringBuilder();
    State cur = goalState;
    while (cur != null) {
      MoveParent mp = parents.get(cur);
      if (mp == null) break;
      if (mp.move != null) path.append(mp.move);
      cur = mp.prev;
    }
    path.reverse();

    double totalCost = gScore.get(goalState);
    return new SearchOutcome(path.toString(), totalCost,
iterations, elapsedMs, exploration);
  }

  private static double priority(
      AlgorithmKind algo,
      HeuristicKind heur,
      int[] goals,
      Map<Integer, int[]> digitPos,
      int minPc,
      int goalR,
      int goalC,
      State st,
      double g) {
    double h =
        IceLogic.heuristic(heur, st.r, st.c, st.phase, goals,
digitPos, goalR, goalC, minPc);
    switch (algo) {
      case UCS:
        return g;
      case GBFS:
        return h;
      case ASTAR:
        return g + h;
      default:
        throw new IllegalArgumentException("Algorithm");
    }
```

```java
  }

  private static final class PQNode {
    final double priority;
    final int tie;
    final State state;
    final double g;

    PQNode(double priority, int tie, State state, double g) {
      this.priority = priority;
      this.tie = tie;
      this.state = state;
      this.g = g;
    }
  }
}
```

e. InputParser.java

```java
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public final class InputParser {

  private InputParser() {}

  public static ProblemInstance parse(String text) {
    List<String> raw = new ArrayList<>();
    for (String line : text.split("\\R")) {
      if (!line.strip().isEmpty()) raw.add(line);
    }
    if (raw.isEmpty()) throw new IllegalArgumentException("File
kosong.");

    String[] head = raw.get(0).trim().split("\\s+");
    if (head.length != 2) throw new
IllegalArgumentException("Baris pertama harus berisi N M.");
```

```
    int n = Integer.parseInt(head[0]);
    int m = Integer.parseInt(head[1]);
    if (n <= 0 || m <= 0) throw new IllegalArgumentException("N
dan M harus positif.");

    int needLines = 1 + 2 * n;
    if (raw.size() < needLines) {
      throw new IllegalArgumentException(
          "Diharapkan setidaknya " + needLines + " baris
non-kosong, dapat " + raw.size() + ".");
    }

    List<String> gridLines = raw.subList(1, 1 + n);
    List<String> costLines = raw.subList(1 + n, 1 + 2 * n);

    for (int i = 0; i < n; i++) {
      String row = gridLines.get(i);
      if (row.length() != m) throw new
IllegalArgumentException("Panjang baris grid tidak konsisten
dengan M.");
    }

    int[][] costs = new int[n][m];
    for (int i = 0; i < n; i++) {
      String[] parts = costLines.get(i).trim().split("\\s+");
      if (parts.length != m)
        throw new IllegalArgumentException("Baris cost " + (i + 1)
+ " harus berisi " + m + " angka.");
      for (int j = 0; j < m; j++) costs[i][j] =
Integer.parseInt(parts[j]);
    }

    char[][] g = new char[n][m];
    for (int i = 0; i < n; i++) g[i] =
gridLines.get(i).toCharArray();

    Integer startR = null, startC = null, goalR = null, goalC =
null;
    Map<Integer, int[]> digitPos = new LinkedHashMap<>();
```

```
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
        char ch = g[i][j];
        if (ch == 'Z') {
          if (startR != null) throw new
IllegalArgumentException("Lebih dari satu Z.");
          startR = i;
          startC = j;
          g[i][j] = '*';
        } else if (ch == 'O') {
          if (goalR != null) throw new
IllegalArgumentException("Lebih dari satu O.");
          goalR = i;
          goalC = j;
          g[i][j] = '*';
        } else if (ch >= '0' && ch <= '9') {
          int d = ch - '0';
          if (digitPos.containsKey(d)) throw new
IllegalArgumentException("Digit " + d + " muncul lebih dari
sekali.");
          digitPos.put(d, new int[] {i, j});
        } else if (ch != '*' && ch != 'X' && ch != 'L') {
          throw new IllegalArgumentException("Karakter tidak
dikenal di (" + i + "," + j + "): '" + ch + "'");
        }
      }
    }

    if (startR == null || goalR == null) throw new
IllegalArgumentException("Harus ada tepat satu Z dan satu O.");

    String[] grid = new String[n];
    for (int i = 0; i < n; i++) grid[i] = new String(g[i]);

    int[] goals;
    if (digitPos.isEmpty()) goals = new int[0];
    else {
      List<Integer> keys = new ArrayList<>(digitPos.keySet());
```

```
      keys.sort(Integer::compareTo);
      int lo = keys.get(0);
      int hi = keys.get(keys.size() - 1);
      for (int d = lo; d <= hi; d++) {
        if (!digitPos.containsKey(d)) {
          throw new IllegalArgumentException(
              "Digit pada papan harus berurutan tanpa celah dari "
+ lo + " sampai " + hi + ".");
        }
      }
      goals = new int[hi - lo + 1];
      for (int k = 0; k < goals.length; k++) goals[k] = lo + k;
    }

    return new ProblemInstance(grid, costs, startR, startC, goalR,
goalC, goals);
  }
}
```

f. MainCLI.java

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Locale;
import java.util.Scanner;


public final class MainCli {

  private MainCli() {}

  public static void main(String[] args) throws IOException {
    Scanner sc = new Scanner(System.in);

    System.out.println(">> Masukan file input (.txt):");
    String pathStr = stripQuotes(sc.nextLine().trim());
```

```
    if (pathStr.isEmpty()) {
      System.err.println("Path kosong.");
      System.exit(1);
    }
    Path path = Path.of(pathStr);
    if (!Files.isRegularFile(path)) {
      System.err.println("File tidak ditemukan: " + pathStr);
      System.exit(1);
    }

    String raw = Files.readString(path, StandardCharsets.UTF_8);

    ProblemInstance prob;
    try {
      prob = InputParser.parse(raw);
    } catch (IllegalArgumentException e) {
      System.err.println("Input tidak valid: " + e.getMessage());
      System.exit(1);
      return;
    }

    System.out.println(">> Pilih Algoritma (UCS/GBFS/A*)");
    String algoStr =
sc.nextLine().trim().toUpperCase(Locale.ROOT);
    AlgorithmKind algo;
    if ("UCS".equals(algoStr)) algo = AlgorithmKind.UCS;
    else if ("GBFS".equals(algoStr)) algo = AlgorithmKind.GBFS;
    else if ("A*".equals(algoStr) || "ASTAR".equals(algoStr)) algo
= AlgorithmKind.ASTAR;
    else {
      System.err.println("Algoritma harus UCS, GBFS, atau A*.");
      System.exit(1);
      return;
    }

    HeuristicKind heur = HeuristicKind.H1;
    if (algo != AlgorithmKind.UCS) {
      System.out.println(">> Heuristic apa yang anda pilih?
(H1/H2/H3)");
```

```
      String hStr = sc.nextLine().trim().toUpperCase(Locale.ROOT);
      if ("H1".equals(hStr)) heur = HeuristicKind.H1;
      else if ("H2".equals(hStr)) heur = HeuristicKind.H2;
      else if ("H3".equals(hStr)) heur = HeuristicKind.H3;
      else {
        System.err.println("Heuristic harus H1, H2, atau H3.");
        System.exit(1);
        return;
      }
    }

    SearchOutcome out = IceSearcher.search(prob, algo, heur);

    if (out.path == null) {
      System.out.println("Tidak ada solusi.");
      System.out.printf(Locale.ROOT, ">> Waktu eksekusi: %.2f
ms%n", out.elapsedMs);
      System.out.println(">> Banyak iterasi yang dilakukan: " +
out.iterations);
      return;
    }

    System.out.println();
    System.out.println("Solusi Yang Ditemukan : " + out.path);
    if (Math.abs(out.totalCost - Math.round(out.totalCost)) <
1e-6)
      System.out.println("Cost dari Solusi : " +
Math.round(out.totalCost));
    else System.out.printf(Locale.ROOT, "Cost dari Solusi : %s%n",
out.totalCost);

    List<String[]> frames = IceLogic.applyMoves(prob, out.path);
    System.out.println("\nInitial");
    printBoard(frames.get(0));
    for (int i = 1; i < frames.size(); i++) {
      char mv = out.path.charAt(i - 1);
      System.out.println();
      System.out.println("Step " + i + " : " + mv);
      printBoard(frames.get(i));
```

```
    }

    System.out.printf(Locale.ROOT, "%n>> Waktu eksekusi: %.2f
ms%n", out.elapsedMs);
    System.out.println(">> Banyak iterasi yang dilakukan: " +
out.iterations);

    System.out.println("\n>> Apakah Anda ingin melakukan playback?
(Ya/Tidak):");
    if (yes(sc.nextLine())) {
      System.out.println(">> Pada step berapa anda ingin memulai
playback:");
      int startStep = 0;
      try {
        startStep = Integer.parseInt(sc.nextLine().trim());
      } catch (NumberFormatException ignored) {
      }
      System.out.println(">> Playback solusi (1) atau eksplorasi
node (2)? [1/2]");
      String mode = sc.nextLine().trim();
      if ("2".equals(mode)) {
        Playback.playExploration(out.explorationOrder, prob,
startStep, sc);
      } else Playback.playSolution(frames, startStep, sc);
    }

    System.out.println("\n>> Apakah Anda ingin menyimpan solusi?
(Ya/Tidak):");
    if (yes(sc.nextLine())) {
      System.out.println(">> Path file output solusi:");
      String outp = stripQuotes(sc.nextLine().trim());
      String algoOut = algo == AlgorithmKind.ASTAR ? "A*" :
algo.name();
      StringBuilder sb = new StringBuilder();
      sb.append("path ").append(out.path).append('\n');
      sb.append("cost
").append(formatCost(out.totalCost)).append('\n');
      sb.append("algorithm ").append(algoOut).append('\n');
      sb.append("heuristic ").append(heur.name()).append('\n');
```

```
      sb.append("iterations
").append(out.iterations).append('\n');
      sb.append(String.format(Locale.ROOT, "time_ms %.6f%n%n",
out.elapsedMs));
      for (int i = 0; i < frames.size(); i++) {
        sb.append("step ").append(i).append('\n');
        sb.append(joinBoard(frames.get(i))).append("\n\n");
      }
      Files.writeString(Path.of(outp), sb.toString(),
StandardCharsets.UTF_8);
      System.out.println("Solusi disimpan pada " + outp);
    }

    System.out.println("\n>> Simpan log iterasi ke .txt?
(Ya/Tidak):");
    if (yes(sc.nextLine())) {
      System.out.println(">> Path file log iterasi:");
      String logp = stripQuotes(sc.nextLine().trim());
      Playback.writeIterationLog(Path.of(logp),
out.explorationOrder);
      System.out.println("Log disimpan pada " + logp);
    }
  }

  private static String formatCost(double c) {
    if (Math.abs(c - Math.round(c)) < 1e-6) return
Long.toString(Math.round(c));
    return Double.toString(c);
  }

  private static void printBoard(String[] rows) {
    System.out.println(joinBoard(rows));
  }

  private static String joinBoard(String[] rows) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < rows.length; i++) {
      sb.append(rows[i]);
      if (i + 1 < rows.length) sb.append('\n');
```

```java
    }
    return sb.toString();
  }

  private static boolean yes(String line) {
    String s = line.strip().toLowerCase(Locale.ROOT);
    return s.equals("ya") || s.equals("y") || s.equals("yes");
  }

  private static String stripQuotes(String s) {
    if (s.length() >= 2 && ((s.startsWith("\"") &&
s.endsWith("\"")) || (s.startsWith("'") && s.endsWith("'")))) {
      return s.substring(1, s.length() - 1);
    }
    return s;
  }
}
```

g. MoveParent.java

```java
public final class MoveParent {
  public final State prev;
  public final Character move;

  public MoveParent(State prev, Character move) {
    this.prev = prev;
    this.move = move;
  }
}
```

h. PlayBack.java

```java
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
```

```java
import java.util.Locale;
import java.util.Scanner;

public final class Playback {

  private Playback() {}

  public static void playSolution(List<String[]> frames, int
startStep, Scanner sc) {
    int idx = Math.max(0, Math.min(startStep, frames.size() - 1));
    System.out.println();
    System.out.println(
        "Playback solusi: ketik `a` atau `>` maju, `d` atau `<`
mundur, `w` lompat ke step, `q` keluar.");
    while (true) {
      System.out.printf(Locale.ROOT, "%n--- Step %d / %d ---%n",
idx, frames.size() - 1);
      System.out.println(String.join(System.lineSeparator(),
frames.get(idx)));

      System.out.print("> ");
      String line = sc.nextLine().trim().toLowerCase(Locale.ROOT);
      if (line.isEmpty()) continue;
      char c = line.charAt(0);
      if (c == 'q') break;
      if (c == 'a' || c == '>') idx = Math.min(frames.size() - 1,
idx + 1);
      else if (c == 'd' || c == '<') idx = Math.max(0, idx - 1);
      else if (c == 'w') {
        System.out.print("Lompat ke step (angka): ");
        try {
          int j = Integer.parseInt(sc.nextLine().trim());
          idx = Math.max(0, Math.min(j, frames.size() - 1));
        } catch (NumberFormatException ignored) {
        }
      }
    }
  }
```

```java
  public static void playExploration(
      List<State> exploration, ProblemInstance p, int startStep,
Scanner sc) {
    if (exploration.isEmpty()) {
      System.out.println("Tidak ada data eksplorasi.");
      return;
    }
    int n = p.n(), m = p.m();
    List<String[]> frames = new ArrayList<>(exploration.size());
    for (State st : exploration) {
      frames.add(IceLogic.renderExplorationState(p.grid, p.goalR,
p.goalC, st, p.goals, n, m));
    }
    playSolution(frames, startStep, sc);
  }

  public static void writeIterationLog(Path path, List<State>
exploration) throws IOException {
    StringBuilder sb = new StringBuilder();
    sb.append("total_iterations
").append(exploration.size()).append('\n');
    int i = 0;
    for (State st : exploration) {
      sb.append("iter ")
          .append(i++)
          .append(" row ")
          .append(st.r)
          .append(" col ")
          .append(st.c)
          .append(" phase ")
          .append(st.phase)
          .append('\n');
    }
    Files.writeString(path, sb.toString(),
StandardCharsets.UTF_8);
  }
}
```

i. ProblemInstance.java

```java
public final class ProblemInstance {
  public final String[] grid;
  public final int[][] costs;
  public final int startR;
  public final int startC;
  public final int goalR;
  public final int goalC;
  public final int[] goals;

  public ProblemInstance(
      String[] grid,
      int[][] costs,
      int startR,
      int startC,
      int goalR,
      int goalC,
      int[] goals) {
    this.grid = grid;
    this.costs = costs;
    this.startR = startR;
    this.startC = startC;
    this.goalR = goalR;
    this.goalC = goalC;
    this.goals = goals;
  }

  public int n() {
    return grid.length;
  }

  public int m() {
    return grid[0].length();
  }
}
```

j. SearchOutcome.java

```java
import java.util.List;
```

```java
/** Result of pathfinding: path may be null if unsolvable. */
public final class SearchOutcome {
  public final String path;
  public final double totalCost;
  public final int iterations;
  public final double elapsedMs;
  public final List<State> explorationOrder;

  public SearchOutcome(
      String path, double totalCost, int iterations, double
elapsedMs, List<State> explorationOrder) {
    this.path = path;
    this.totalCost = totalCost;
    this.iterations = iterations;
    this.elapsedMs = elapsedMs;
    this.explorationOrder = explorationOrder;
  }
}
```

k. SlideResult.java

```java
public final class SlideResult {
  public final int endR;
  public final int endC;
  public final int newPhase;
  public final int moveCost;

  public SlideResult(int endR, int endC, int newPhase, int
moveCost) {
    this.endR = endR;
    this.endC = endC;
    this.newPhase = newPhase;
    this.moveCost = moveCost;
  }
}
```

l. State.java

```java
import java.util.Objects;

public final class State {
  public final int r;
  public final int c;
  public final int phase;

  public State(int r, int c, int phase) {
    this.r = r;
    this.c = c;
    this.phase = phase;
  }

  @Override
  public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof State)) return false;
    State s = (State) o;
    return r == s.r && c == s.c && phase == s.phase;
  }

  @Override
  public int hashCode() {
    return Objects.hash(r, c, phase);
  }
}
```

Bab 4

Tangkapan Layar

a. Input 1

```
4 5
XXXXX
XZ**X
X**0X
XXXXX
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

```
File tidak ditemukan: test/Input1
rino@LAPTOP-ESI19P0R:/mnt/d/STIMA/Tucil3/Tucil3_13524138$ ./bin/run-java.sh
>> Masukan file input (.txt):
test/Input1.txt
>> Pilih Algoritma (UCS/GBFS/A*)
ucs

Solusi Yang Ditemukan : DR
Cost dari Solusi : 3

Initial
XXXXX
XZ**X
X**0X
XXXXX

Step 1 : D
XXXXX
X***X
XZ*0X
XXXXX

Step 2 : R
XXXXX
X***X
X**ZX
XXXXX

>> Waktu eksekusi: 10.80 ms
>> Banyak iterasi yang dilakukan: 4
```

b. Input 2

```
7 7
XXXXXXX
X0***0X
X*****X
XZ***1X
X*****X
X*****X
XXXXXXX
10 10 10 10 10 10 10
10 10 10 10 10 10 10
10 10 10 10 10 10 10
10 10 10 10 10 10 10
10 10 10 10 10 10 10
10 10 10 10 10 10 10
10 10 10 10 10 10 10
```

```
rino@LAPTOP-ESI19P0R:/mnt/d/STIMA/Tucil3/Tucil3_13524138$ ./bin/run-java.sh
>> Masukan file input (.txt):
test/Input2.txt
>> Pilih Algoritma (UCS/GBFS/A*)
ucs

Solusi Yang Ditemukan : URDU
Cost dari Solusi : 140

Initial
XXXXXXX
X0***0X
X*****X
XZ***1X
X*****X
X*****X
XXXXXXX

Step 1 : U
XXXXXXX
XZ***0X
X*****X
X****1X
X*****X
X*****X
XXXXXXX
```

```
Step 2 : R
XXXXXXX
X****ZX
X*****X
X****1X
X*****X
X*****X
XXXXXXX

Step 3 : D
XXXXXXX
X****0X
X*****X
X*****X
X*****X
X****ZX
XXXXXXX

Step 4 : U
XXXXXXX
X****ZX
X*****X
X*****X
X*****X
X*****X
XXXXXXX

>> Waktu eksekusi: 7.23 ms
>> Banyak iterasi yang dilakukan: 10
```

c. Input 3

```
6 8
XXXXXXXX
X0***X0X
X*X***1X
X*X*X**X
XZ*****X
XXXXXXXX
9 9 9 9 9 9 9 9
9 2 3 4 5 9 7 9
9 2 9 6 4 3 8 9
9 3 9 5 9 2 1 9
9 1 2 2 2 2 2 9
9 9 9 9 9 9 9 9
```

```
>> Masukan file input (.txt):
test/Input3.txt
>> Pilih Algoritma (UCS/GBFS/A*)
ucs

Solusi Yang Ditemukan : UDRU
Cost dari Solusi : 39

Initial
XXXXXXXX
X0***XOX
X*X***1X
X*X*X**X
XZ*****X
XXXXXXXX

Step 1 : U
XXXXXXXX
XZ***XOX
X*X***1X
X*X*X**X
X******X
XXXXXXXX

Step 2 : D
XXXXXXXX
X****XOX
X*X***1X
X*X*X**X
XZ*****X
XXXXXXXX
```

```
Step 3 : R
XXXXXXXX
X****XOX
X*X***1X
X*X*X**X
X*****ZX
XXXXXXXX

Step 4 : U
XXXXXXXX
X****XZX
X*X****X
X*X*X**X
X******X
XXXXXXXX

>> Waktu eksekusi: 11.67 ms
>> Banyak iterasi yang dilakukan: 13
```

d. Input 4

```
7 7
XXXXXXX
XZ****X
X*XL*0X
X***X*X
X0***1X
X*****X
XXXXXXX
9 9 9 9 9 9 9
9 1 2 2 3 4 9
9 2 5 9 2 7 9
9 2 2 2 9 3 9
9 1 3 4 2 6 9
9 2 2 2 2 2 9
9 9 9 9 9 9 9
```

```
>> Masukan file input (.txt):
test/Input4.txt
>> Pilih Algoritma (UCS/GBFS/A*)
ucs
Tidak ada solusi.
>> Waktu eksekusi: 8.27 ms
>> Banyak iterasi yang dilakukan: 10
```

e. Input 5

```
7 7
XXXXXXX
XZ****X
X*XXX*X
X*XOX*X
X*XXX*X
X*****X
XXXXXXX
9 9 9 9 9 9 9
9 1 1 1 1 1 9
9 1 9 9 9 1 9
9 1 9 5 9 1 9
9 1 9 9 9 1 9
9 1 1 1 1 1 9
9 9 9 9 9 9 9
```

```
>> Masukan file input (.txt):
test/Input5.txt
>> Pilih Algoritma (UCS/GBFS/A*)
ucs
Tidak ada solusi.
>> Waktu eksekusi: 12.71 ms
>> Banyak iterasi yang dilakukan: 4
```

Bab 5

Analisis Hasil

a. Analisis Optimasi Solusi (Total Cost)

UCS (Uniform Cost Search) & A*: UCS dan A* pasti menghasilkan Total Cost yang sama dan paling rendah*. Hal ini karena UCS mengeksplorasi berdasarkan g(n) murni, dan A* menggunakan f(n) = g(n) + h(n) dengan heuristik yang admissible. Keduanya dijamin optimal.

- GBFS (Greedy Best-First Search):
- GBFS memiliki kemungkinan besar untuk menghasilkan Total Cost lebih besar (sub-optimal). Karena GBFS hanya berfokus pada jarak terdekat menuju tujuan f(n) = h(n), algoritma ini akan menerobos jalur yang berbiaya mahal asalkan jalur tersebut membuat karakter secara posisi terlihat lebih dekat dengan tujuan.

2. Analisis Jumlah Iterasi (Eksplorasi Node)

- GBFS:

  GBFS akan mencetak jumlah iterasi yang paling sedikit. Algoritma ini bersifat serakah dan langsung bergerak ke arah yang mendekatkan jarak ke target, sehingga tidak banyak membuang waktu mengeksplorasi jalur alternatif.
- UCS:

  UCS akan mencetak jumlah iterasi yang sangat tinggi. Algoritma ini memperluas pencarian ke segala arah karena tidak memiliki informasi mengenai tujuan (heuristik) untuk mengarahkannya.
- A:

  A* akan mencetak jumlah iterasi di tengah-tengah antara GBFS dan UCS. Adanya heuristik membantu memotong iterasi pada jalur-jalur yang mahal secara g(n) dan jauh secara h(n)

3. Analisis Pengaruh Fase (Multiple Targets)

- UCS: Pencarian UCS akan mengalami pelambatan eksponensial karena untuk setiap fase, ia kembali melakukan blind search.
- A*: Dengan menggunakan heuristik (seperti H2 atau H3), Pencarian A* dapat mengakumulasi sisa jarak dari posisi saat ini -> sisa angka target -> garis finish (O). Oleh karena itu, iterasi A* akan jauh lebih rapi dan terarah pada tahapan-tahapan ini.

Kesimpulan Analisis

Dari pengerjaan program ini, dapat disimpulkan bahwa Algoritma A* adalah algoritma terbaik secara keseluruhan untuk menyelesaikan sliding ice puzzle ini. A* tidak mengorbankan rute optimal, tetapi juga meminimalisir waktu komputasi dan penggunaan memori seperti algoritma GBFS. GBFS cocok digunakan jika kecepatan adalah prioritas dan cost dari pergerakan tidak penting.

Lampiran

Pernyataan Karya Orisinalitas

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Ahmad Rinofaros Mochtar